

System Architecture Document

LKA System — Team Equinox

aBAJA SAEINDIA 2026

aligned with the formatting, technical depth, and structure of your sample file, but customized for Team Equinox.

System Architecture Document

LKA System — Team Equinox | aBAJA SAEINDIA 2026

1. System Overview

This document describes the software architecture of the Lane Keep Assist (LKA) system developed by Team Equinox for the aBAJA SAEINDIA 2026 Software Virtual World Simulation event. The system operates within IPG CarMaker 15.0 and interfaces via the CarMaker Python APO (Asynchronous Process Object). The perception stack relies exclusively on a Camera RSI sensor; the system does not utilize ground-truth lane data or DVA-based lane quantities.

Key specifications: | Parameter | Value | | :--- | :--- | | Simulator | IPG CarMaker 15.0 | | Rendering | MovieNX | | Camera sensor | Camera RSI — 640×480 RGB, ~30 Hz over TCP port 2210 | | Target speed | 30 km/h | | Control loop rate | 30 Hz | | Vehicle model | Demo_IPG_CompanyCar (wheelbase 2.622 m, mass 1 391 kg) | | Language | Python 3 with OpenCV, NumPy, IPG CarMaker Python API |

2. Module Architecture

The architecture is decomposed into five specialized Python modules. Following a strict downward dependency flow, the main loop orchestrates the execution while sub-modules remain decoupled to ensure modularity and ease of unit testing.

	Role	Key Outputs
A_Main.py**	Main control loop — bridges CarMaker and the perception/control stack.	Steering command (rad)

on_engine.py**	CV module. Applies BEV warp, Sliding Window search, and polynomial fitting.	Lateral error (m), lookahead
y**	Single source of truth for all parameters: PID gains, BEV coordinates, and thresholds.	Constants (imported)
_utils.py**	Utility module: RMSE calculation, DNF watchdog, and CSV telemetry logging.	RMSE value, DNF flag, lka\
nt.py**	TCP socket client for Camera RSI stream on localhost:2210. Handles decoding.	Raw BGR frame (NumPy ar

3. Data Flow

Each control cycle (nominally 1/30 s) executes the following pipeline:

Step	Module	Description
1	tcp_client	Receive raw 640×480 RGB frame from Camera RSI over TCP
2	perception_engine	Convert to grayscale → apply binary thresholding
3	perception_engine	Warp to Bird's-Eye View (source trapezoid → top-down rectangle)
4	perception_engine	Bottom-half histogram → find left/right lane base positions
5	perception_engine	Sliding window search upward → collect lane pixel coordinates
6	perception_engine	Fit 1st-order polynomial to each lane → compute lane centre
7	perception_engine	Compute lateral error = lane centre – image centre (metric)
8	perception_engine	Compute lookahead offset at 5.0 m ahead on fitted lane
9	Equinox_Main	Pure Pursuit: compute baseline steering angle from lookahead geometry
10	Equinox_Main	PID: compute correction term from lateral error
11	Equinox_Main	Sum and rate-limit → clamp to ±0.20 rad road-wheel angle
12	Equinox_Main	Write road-wheel angle to CarMaker APO (DVA write)
13	equinox_utils	Log telemetry to CSV; update RMSE; check DNF status

4. Perception Pipeline — BEV + Sliding Window

The Team Equinox perception approach utilizes a classical computer vision pipeline. After evaluating high-complexity filters, a robust BEV + Sliding Window method was chosen for its reliability in varying CarMaker light conditions.

4.1 Bird's-Eye View Transform

A perspective warp maps a trapezoidal region of the front-facing camera to a top-down rectangular plane. This eliminates perspective foreshortening, ensuring lane lines appear as parallel vertical strips, which simplifies mathematical modeling of the lateral error.

4.2 Sliding Window Lane Search

The algorithm utilizes a bottom-half column histogram to identify lane origins. A stack of 9 windows (± 50 px margin) searches upward, re-centering on the centroid of white pixels. This method allows the system to bridge gaps in dashed lines and track sharp curves effectively.

4.3 Polynomial Fitting and Gap Bridging

A first-order polynomial is fitted to detected pixels. To handle intermittent occlusions or lighting glares, the system caches the last 15 frames of valid polynomial fits, allowing the vehicle to "blind steer" through brief sensor dropouts without losing stability.

5. Control Architecture — Pure Pursuit + PID

Team Equinox utilizes a hybrid controller. While a PID controller manages immediate lateral error, the Pure Pursuit geometry provides a "look-ahead" capability to anticipate upcoming curves.

Parameter	Value	Rationale
Lookahead distance	5.0 m	Provides ~ 0.6 s preview at 30 km/h; balances smoothness.
Vehicle wheelbase	2.622 m	Matched to Demo\IPG_CompanyCar specification.
Steering ratio	16:1	Maps internal road-wheel angle to virtual steering wheel.
Max wheel angle	± 0.20 rad	Physical steering limit enforced in software to prevent spin-out.
Rate limit	0.03 rad/f	Prevents jerky actuator motion and maintains tire grip.
PID (Kp, Ki, Kd)	0.1, 0.005, 0.08	Tuned to eliminate steady-state offset on straightaways.

6. Safety Mechanisms

Mechanism	Implementation	Behaviour
DNF watchdog	equinox_utils.py	Triggers DNF if front axle exits lane for >5 consecutive seconds.
Steering rate limiter	Equinox_Main.py	Caps steering delta at 0.03 rad per frame to ensure stability.
Angle clamp	Equinox_Main.py	Final road-wheel command is strictly clamped to ± 0.20 rad.
Fit cache fallback	perception_engine	Uses last valid fit for up to 15 frames if lane detection is lost.
TCP Reconnect	tcp_client.py	Automatically reconnects to RSI stream on socket drops.

7. Logged Outputs

All run data is stored in results/EQUINOX_LKA/:

File	Description
Data/Logs/lka_run.csv	Per-frame telemetry: timestamp, error, steering angle, speed.

debug_frames/	Folder containing BEV view and sliding window visualizations.
debug_raw_frame.png	The final raw camera frame captured during the run.
debug_bev.png	Perspective-corrected Bird's-Eye View frame.
debug_vision.png	Annotated front-view frame with lane overlays and error metrics.